

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 10

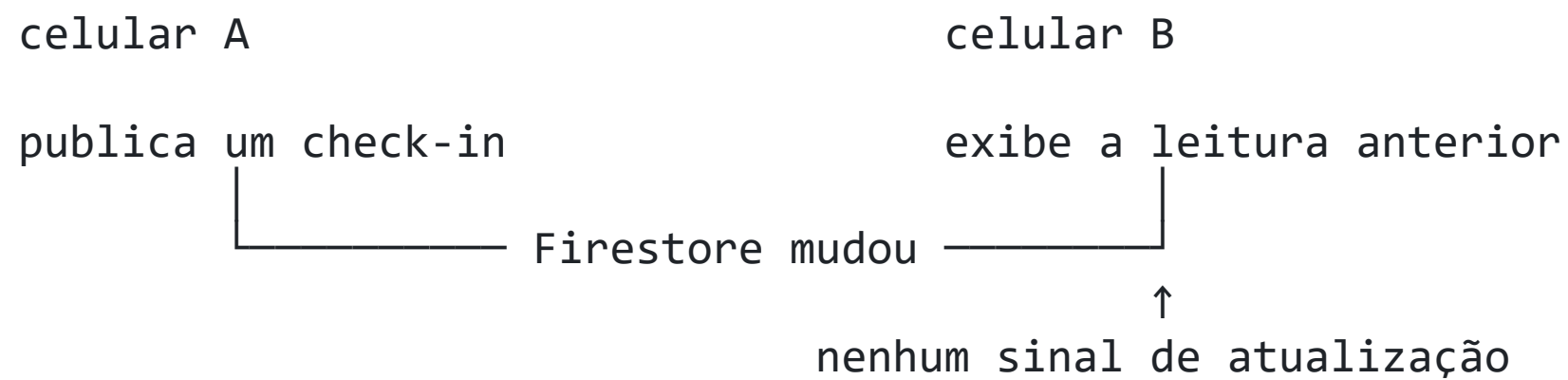
Dados que mudam em tempo real

`onSnapshot` e ciclo de vida da tela

UMA PESSOA PUBLICA EM UM CELULAR

POR QUE O OUTRO CONTINUA DESATUALIZADO?

DOIS CLIENTES, DUAS FOTOGRAFIAS



O BANCO MUDOU. O CLIENTE B NÃO SABE DISSO.

UMA LEITURA PONTUAL NÃO RECEBE MUDANÇAS FUTURAS

cliente B abre a tela



getDocs



exibe uma fotografia dos dados



o banco muda, mas a tela não consulta novamente

Para acompanhar mudanças, o cliente precisa manter uma assinatura.

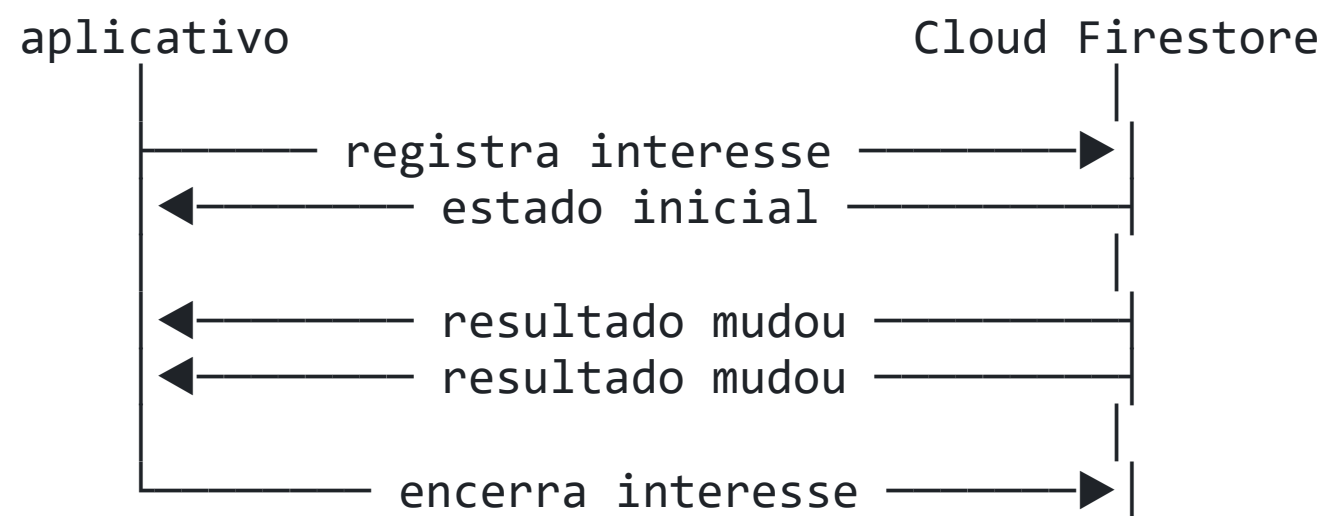
OBJETIVOS

- diferenciar `getDocs` de `onSnapshot` ;
- implementar um listener que atualiza a tela;
- encerrar o listener quando a tela perde o foco;
- tratar a falha do listener.

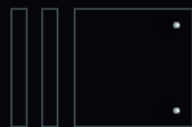
LEITURA PONTUAL OU ASSINATURA

Necessidade	API
carregar os dados uma vez	<code>getDocs</code>
acompanhar mudanças enquanto a tela estiver ativa	<code>onSnapshot</code>

ASSINATURA ORIENTADA A EVENTOS

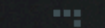


⁺ O cliente mantém interesse ativo enquanto os dados forem necessários.



A CONSULTA CONTINUA SENDO A MESMA

O MODO DE CONSUMO MUDA.



O CONJUNTO OBSERVADO

```
const recentPostsQuery = query(  
  collection(db, "posts"),  
  orderBy("createdAt", "desc"),  
  limit(10),  
);
```

Essa `Query` define:

- a coleção;
- a ordenação;
- o limite do resultado.

Ela ainda não executou uma leitura.

DOIS CONTRATOS PARA A MESMA QUERY

Operação	Contrato
<code>getDocs(recentPostsQuery)</code>	produz uma fotografia e termina
<code>onSnapshot(recentPostsQuery)</code>	registra callbacks e permanece ativo

<code>getDocs</code>	pedir – aguardar – receber – terminar
<code>onSnapshot</code>	assinar – receber – receber – ... – cancelar

LEITURA PONTUAL

```
const snapshot = await getDocs(recentPostsQuery);  
  
const posts = snapshot.docs.map(mapPost);
```

retorno de getDocs
↓
Promise<QuerySnapshot>

A Promise resolve com o resultado daquela execução.

LEITURA CONTÍNUA

```
const unsubscribe = onSnapshot(  
  recentPostsQuery,  
  (snapshot) => {  
    const posts = snapshot.docs.map(mapPost);  
    console.log(posts);  
  },  
);
```

+
|
retorno de onSnapshot
↓
Unsubscribe

Os dados chegam ao callback. O retorno encerra a assinatura.

O CONTRATO DO LISTENER

```
onSnapshot(query, onNext, onError)
```

recebe a falha terminal

recebe cada QuerySnapshot

define o resultado observado

retorno: Unsubscribe

`onSnapshot` não devolve os documentos e não produz uma Promise.

A PRIMEIRA EMISSÃO

Ao registrar o listener, o callback recebe o estado atual da consulta.

```
Firestore já contém  
[A, B, C]
```

```
listener inicia  
  ↓  
primeiro snapshot  
[A, B, C]
```

NÃO PRECISAMOS EXECUTAR UM GETDOCS ANTES DO LISTENER.

EMISSÕES POSTERIORES

O callback recebe outro `QuerySnapshot` quando o resultado muda.

```
snapshot 1  
[A, B, C]
```

```
documento D é criado
```

```
snapshot 2  
[D, A, B, C]
```

Mudanças relevantes incluem documentos adicionados, modificados ou removidos.

POR QUANTO TEMPO O LISTENER DEVE PERMANECER ATIVO?

Uma assinatura também ocupa recursos quando ninguém observa a tela.

ENCERRAMENTO DA ASSINATURA

```
const unsubscribe = onSnapshot(  
  recentPostsQuery,  
  handleSnapshot,  
  handleError,  
);  
  
// Quando os dados deixarem de ser necessários  
unsubscribe();
```

O cancelamento interrompe callbacks futuros e permite que o cliente pare de receber atualizações.

SETUP E CLEANUP

```
setup
└─ inicia a assinatura

cleanup
└─ encerra a mesma assinatura
```

O início e o encerramento formam uma única responsabilidade.

CRIAR O LISTENER SEM PLANEJAR O CLEANUP DEIXA O CONTRATO INCOMPLETO.

MONTAGEM E FOCO

Uma tela móvel pode continuar montada quando outra tela fica visível.

Comunidade montada

Comunidade em foco

outra tela em foco

Comunidade ainda montada

Cleanup no unmount pode manter o listener ativo durante a navegação.

DURAÇÃO ESCOLHIDA PARA O MURAL

```
Comunidade ganha foco
↓
iniciar listener
↓
receber snapshots
↓
Comunidade perde foco
↓
cancelar listener
```

A ASSINATURA ACOMPANHA A VISIBILIDADE DA TELA.

USEFOCUSEFFECT

```
useFocusEffect(  
  useCallback(() => {  
    return onSnapshot(  
      recentPostsQuery,  
      handleSnapshot,  
      handleError,  
    );  
  }, []),  
);
```

○ Expo Router executa o setup no foco e o cleanup quando a rota perde o foco.

CALLBACK DE ERRO

```
const unsubscribe = onSnapshot(
  recentPostsQuery,
  (snapshot) => {
    onData(snapshot.docs.map(mapPost));
  },
  (error) => {
    onError(error);
  },
);
```

Permissão insuficiente e consulta inválida podem encerrar o listener.

Depois da falha entregue pelo Firestore, o listener não emite novos eventos.

HOOK DE ASSINATURA

```
export function useRecentPostsSubscription() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [error, setError] = useState<unknown>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(
    useCallback(() => {
      setError(null);
      setIsLoading(true);

      return onSnapshot(
        recentPostsQuery,
        (snapshot) => {
          setPosts(snapshot.docs.map(mapPost));
          setIsLoading(false);
        },
        (cause) => {
          setError(cause);
          setIsLoading(false);
        },
      );
    }, []),
  );

  return { posts, error, isLoading };
}
```

TELA CONSUMIDORA

```
const {
  posts,
  error,
  isLoading,
} = useRecentPostsSubscription();

if (isLoading) {
  return <LoadingState />;
}

if (error) {
  return <ErrorState />;
}

return <PostList posts={posts} />;
```

A tela renderiza estados. Ela não conhece `onSnapshot`.

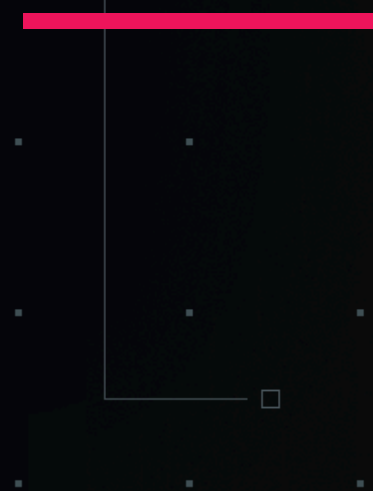
SÍNTESE

- `getDocs` produz uma fotografia e termina;
- `onSnapshot` entrega o estado inicial e permanece ativo;
- cada snapshot contém o resultado atual da consulta;
- o callback de dados atualiza a tela;
- o callback de erro informa uma falha;
- `Unsubscribe` encerra o recurso;
- o foco da tela delimita a assinatura.



UMA ASSINATURA POSSUI COMEÇO, EMISSÕES, FALHA E ENCERRAMENTO

IGNORAR QUALQUER UMA DESSAS ETAPAS PRODUZ UMA INTERFACE
INCONSISTENTE.





APLICATIVO “PACEUP”

+ Mural de treinos sincronizado com Cloud Firestore

O QUE VOCÊ VAI APRENDER?

- reproduzir a desatualização entre dois clientes;
- substituir `getDocs` por uma assinatura;
- tratar a primeira emissão e as mudanças posteriores;
- cancelar o listener quando a tela perder foco;
- mostrar uma mensagem quando o listener falhar.

REFERÊNCIAS

- [Cloud Firestore: atualizações em tempo real](#)
- [Firebase JavaScript API: Firestore](#)
- [Expo Router: useFocusEffect](#)

LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026